

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

Направление	09.03.04 Программная инженерия
Профиль	Разработка программно-информационных систем
Факультет	КТИ
Кафедра	МО ЭВМ

К защите допустить

Зав. кафедрой

А.А. Лисс

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА**

**Тема: РАЗРАБОТКА АСИНХРОННОГО МЕТОДА ОТПРАВКИ
ЭЛЕКТРОННЫХ ПИСЕМ В RNR-ПРИЛОЖЕНИЯХ**

Студент		_____	А.А. Денежный
Руководитель	к.т.н., доцент	_____	К.А. Борисенко
Консультант	к.э.н.	_____	О.С. Артамонова

Санкт-Петербург

2024

ЗАДАНИЕ

Утверждаю

Зав. кафедрой МО ЭВМ

А.А. Лисс

« » 2024 г.

Группа 0303

Тема работы: Разработка асинхронного метода отправки электронных писем
в RНР-приложениях

Место выполнения ВКР: Санкт-Петербургский государственный электро-
технический университет «ЛЭТИ» им. В.И. Ульянова (Ленина)

Исходные данные (технические требования):

Асинхронный метод отправки электронных писем в РНР-приложениях

Содержание ВКР:

Введение, Анализ предметной области, Формулировка требований к решению и постановка задачи, Архитектура программной реализации, Исследование разработанного метода, Обеспечение качества разработки, Заключение.

Перечень отчетных материалов: пояснительная записка, иллюстративный материал.

Дополнительные разделы: Обеспечение качества разработки, продукции, программного продукта.

Дата представления ВКР к защите

« 18 » ИЮНЯ 2024 Г.

А.А. Денежный

К.А. Борисенко

КАЛЕНДАРНЫЙ ПЛАН ВЫПОЛНЕНИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Утверждаю

Зав. кафедрой МО ЭВМ

А.А. Лисс

« » 2024 г.

Студент Денежный А.А.

Група 0303

Тема работы: Разработка асинхронного метода отправки электронных писем
в PHP-приложениях

№ п/п	Наименование работ	Срок выполнения
1	Обзор литературы по теме работы	02.04 – 07.04
2	Обзор предметной области	08.04 – 10.04
3	Формулировка требований к решению и постановка задачи	11.04 – 14.04
4	Проектирование архитектуры программной реализации	15.04 – 20.04
5	Разработка программной системы	21.04 – 24.04
6	Обеспечение качества разработки	25.04 – 27.04
7	Оформление пояснительной записки	28.04 – 30.04
8	Оформление иллюстративного материала	30.04 – 01.05
9	Предзащита	27.05

Студент

А.А. Денежный

Руководитель к.т.н., доцент

К.А. Борисенко

РЕФЕРАТ

Пояснительная записка 69 стр., 11 рис., 30 табл., 17 ист.

РНР, асинхронность, отправка электронных писем, брокер сообщений

Объектом исследования являются способы использования асинхронного программирования при разработке РНР-приложений.

Предметом исследования является процесс асинхронной отправки электронных писем.

Целью работы является повышение производительности отправки электронных писем в РНР-приложении Colibri.travel.

В рамках выполнения данной работы было проведено исследование предметной области для определения того, как можно использовать асинхронное программирование в РНР, а также были рассмотрены инструменты для отправки электронных писем в РНР-приложениях. После исследования предметной области был проведен сравнительный обзор аналогов с целью определения их сильных и слабых сторон, потенциальных проблем, связанных с разработкой метода асинхронной отправки электронных писем. После того, как была собрана и обработана вся необходимая информация касательно использования асинхронного программирования и отправки электронных сообщений в РНР-приложениях, было проведено проектирование архитектуры программной реализации и реализована последующая разработка метода. Далее было проведено исследование метода.

ABSTRACT

As part of this work, domain research was conducted to determine how asynchronous programming can be used in PHP, and tools for sending emails in PHP applications were also considered. After researching the subject area, a comparative review of analogues was carried out in order to determine their strengths and weaknesses, and potential problems associated with the development of a method for asynchronously sending emails. After all the necessary information regarding the use of asynchronous programming and sending electronic messages in PHP applications was collected and processed, the architecture of the software implementation was designed and the subsequent development of the method was carried out. Next, a study of the method was carried out.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	9
1 Анализ предметной области.....	11
1.1 Понятие асинхронного программирования	11
1.2 Основные преимущества и недостатки использования асинхронного программирования при разработке веб-приложений	11
1.3 Способы применения асинхронного программирования при разработке PHP-приложений	12
1.3.1 Использование функций и механизмов стандартной библиотеки PHP .	12
1.3.2 Использование сторонних библиотек.....	12
1.3.3 Использование архитектурного подхода «Очередь-Обработчик»	12
1.4 Способы отправки электронных сообщений в PHP-приложениях.....	18
1.4.1 Использование функций стандартной библиотеки PHP.....	12
1.4.2 Использование сторонних библиотек.....	12
1.5 Обзор аналогов.....	20
1.5.1 Принцип отбора аналогов.....	20
1.5.2 Отобранные аналоги.....	20
1.5.3 Критерии сравнения аналогов	20
1.5.4 Сравнение аналогов.....	20
1.5.5 Выводы по итогам сравнения.....	20
2 Формулировка требований к решению и постановка задачи.....	24
2.1 Постановка задачи	24
2.2 Требования к решению	24
2.3 Обоснование выбора метода решения.....	25
3 Архитектура программной реализации.....	26
3.1 Используемые технологии.....	26
3.2 Общая схема работы метода.....	26
3.3 Проектирование новой таблицы в существующей базе данных	28

3.3.1 Обоснование проектирования таблицы.....	28
3.3.2 Определение этапов проектирования таблицы	28
3.3.3 Проектирование таблицы	28
3.4 Конфигурационные файлы	32
3.4.1 Конфигурационный файл queue.php.....	32
3.4.2 Конфигурационный файл eventbus.php	33
3.4.3 Конфигурационный файл worker.php	37
3.5 Структура программной реализации	38
3.5.1 Реализованные классы	41
3.5.2 Взаимодействие классов	45
4 Исследование разработанного метода.....	48
4.1 Исследование времени отправки электронных писем.....	48
4.2 Исследование количества запросов, которое может обработать приложение	52
4.3 Исследование увеличения количества РНР-воркеров	53
4.4 Выводы по итогам исследования разработанного метода.....	55
5 Обеспечение качества разработки, продукции, программного продукта....	56
5.1 Определение потребителей.....	56
5.2 Функции продукции	56
5.3 Качество и характеристики.....	57
5.4 Измерение характеристик качества. Операционное определение.....	58
5.5 Предложения по улучшению.....	59
5.6 Выводы	60
ЗАКЛЮЧЕНИЕ.....	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	63

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ЯП – язык программирования;

Процесс – представляет собой экземпляр запущенной программы в операционной системе;

Поток – представляет собой последовательность выполнения инструкций внутри процесса;

SMTP (Simple Mail Transfer Protocol) – широко используемый сетевой протокол, предназначенный для передачи электронной почты;

HTML (HyperText Markup Language) – язык гипертекстовой разметки;

HTTP (HyperText Transfer Protocol) – сетевой протокол прикладного уровня, предназначенный для передачи гипертекста;

PHP (Hypertext Preprocessor) – скриптовый язык программирования общего назначения, интенсивно применяемый для разработки веб-приложений и веб-сайтов;

AMQP (Advanced Message Queuing Protocol) – открытый протокол прикладного уровня для передачи сообщений между компонентами системы;

ER-модель (Entity Relationship) – модель данных, позволяющая описывать концептуальные схемы предметной области;

API (Application Programming Interface) – описание способа взаимодействия между компьютерными программами;

СУБД – система управления базами данных;

SQL (Structured Query Language) – язык для управления данными в реляционных базах данных;

UML (Unified Modeling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

ВВЕДЕНИЕ

Colibri.travel – онлайн-система бронирования туристических услуг и оформления командировок, разработанная на популярном языке программирования PHP. Отправка электронных писем с выписанными билетами или иными уведомлениями по оформленным услугам клиентам и менеджерам системы – очень частая и важная операция. Язык программирования PHP – один из самых популярных серверных языков программирования. [1] Более 75% веб-сайтов по всему миру разработано именно на этом языке программирования. [2] PHP предлагает большое количество сторонних библиотек и фреймворков, а также предоставляет возможность интеграции с базами данных и другими инструментами для создания сложных веб-приложений и веб-сайтов. [3] Однако PHP по своей природе является синхронным языком программирования. Это означает, что программные операции, реализованные с помощью этого языка программирования, выполняются последовательно, одна за другой. Поэтому каждая следующая операция будет выполнена после полного завершения предыдущей операции. Синхронное выполнение программного кода достаточно просто в освоении для программистов, поэтому большинство веб-приложений и веб-сайтов, созданных с помощью PHP, придерживаются этого подхода. Однако в ситуациях, когда возникает необходимость обрабатывать большое количество пользовательских запросов к веб-приложению или выполнять длительные операции, такие как запросы к базе данных, запросы к API стороннего сервиса или отправка электронных писем на почту, увеличивается время работы приложения.

Таким образом, для улучшения производительности отправки электронных писем в PHP-приложении Colibri.travel, актуальна разработка системы асинхронной отправки электронных сообщений.

Целью работы является повышение производительности отправки электронных писем в PHP-приложении Colibri.travel.

Задачи данной работы:

1. Исследовать способы использования асинхронного программирования в РНР.
2. Рассмотреть асинхронные методы отправки электронных писем в РНР-приложениях.
3. Разработать архитектуру асинхронного метода отправки электронных писем.
4. Реализовать асинхронный метод отправки электронных писем.
5. Исследовать разработанный метод.

Объект исследования – способы использования асинхронного программирования при разработке РНР-приложений.

Предмет исследования – процесс асинхронной отправки электронных писем.

Практическая ценность работы заключается в разработке асинхронного метода отправки электронных писем и внедрения данного метода в уже существующее РНР-приложение Colibri.travel для повышения производительности отправки электронных писем.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Понятие асинхронного программирования

Асинхронность в программировании [4] – это выполнение какого-либо процесса в неблокирующем режиме системного вызова, что позволяет потоку программы продолжить обработку следующих операций. Программный код называется асинхронным, если во время его выполнения какие-либо длительные операции могут быть запущены без ожидания их завершения, и продолжится основной поток выполнения программы.

1.2 Основные преимущества и недостатки использования асинхронного программирования при разработке веб-приложений

Использование асинхронного программирования при разработке веб-приложений обладает рядом преимуществ:

1. Улучшение производительности – данное преимущество обусловлено тем, что основной поток выполнения не дожидается завершения длительных операций и продолжает свою работу. Это позволяет веб-приложениям быстрее обрабатывать запросы пользователей.
2. Повышение надежности – в случае возникновения каких-либо ошибок во время выполнения асинхронного кода, появляется возможность обрабатывать их без остановки основного потока выполнения кода. Также при обработке ошибок можно сохранить необходимые данные и повторно вызвать процесс работы с данными позже.
3. Масштабируемость – при проектировании асинхронных веб-приложений легче добиться способности обрабатывать большое количество запросов при увеличении нагрузки на приложение путем параллеливания задач и распределения нагрузки между несколькими ядрами процессора или серверами.

Однако также существуют недостатки:

1. Увеличение сложности разработки и отладки веб-приложений – по сравнению с синхронным подходом выполнения программного кода

асинхронный подход подразумевает, что выполнение некоторых операций будет происходить непоследовательно, что усложняет понимание использования данного подхода при разработке веб-приложений и увеличивает риск появления ошибок.

2. Увеличение сложности управления состоянием приложения — из-за асинхронной природы операций может возникнуть несогласованность данных во время работы приложения, что приведет к неоднозначному поведению и возникновению непредвиденных ошибок.

Таким образом, использование асинхронного программирования при разработке веб-приложений имеет и преимущества, и недостатки. Однако большинство недостатков возможно избежать путем увеличения профессиональной квалификации программистов и проектирования веб-приложений с учетом специфики асинхронного программирования.

1.3 Способы применения асинхронного программирования при разработке РНР-приложений

1.3.1 Использование функций и механизмов стандартной библиотеки РНР

Язык программирования РНР обладает богатой стандартной библиотекой, которая обеспечивает широкий спектр возможностей: работа с файлами и директориями, работа с базами данных, работа с сетью, обработка строк и массивов, запуск программ и управление процессами в операционной системе и другие.

Функция `exec ()` [5] — это функция, принадлежащая стандартной библиотеке РНР и используемая для выполнения внешней РНР программы. Функция возвращает последнюю строку вывода внешней программы или `false` при возникновении ошибки. Функция использует операционную систему для создания нового процесса, в рамках которого будет выполняться внешняя программа. Аргументы функции `exec ()` представлены в таблице 1.

Таблица 1 – Аргументы функции `exec()`

Тип аргумента	Название аргумента	Описание	Пример
string	<code>\$command</code>	Команда для запуска внешней PHP программы, содержащая путь к программе	<code>php /path/to/script.php</code>
array null	<code>\$output</code>	Массив, содержащий вывод построчный вывод внешней программы	<code>[]</code>
int null	<code>\$result_code</code>	Код статуса выполнения внешней программы	<code>0</code>

Главные недостатки использования функции `exec()` для добавления асинхронности в веб-приложения, разработанные на PHP:

1. Блокировка основного потока выполнения программного кода – основной поток выполнения программного кода остается заблокированным до полного выполнения функции.
2. Истощение лимитов оперативной памяти и процессора сервера – так как `exec()` выполняет внешнюю программу на том же сервере, где выполняется основная программа, то при увеличении нагрузки на основную программу может быть создано неопределенное количество внешних процессов, которые полностью исчерпают лимиты оперативной памяти и процессора сервера, что может привести к падению сервера.
3. Отсутствие контроля выполнения и перезапуска внешней программы – в случае возникновения ошибок во время выполнения. Также обработка ошибок возлагается на внешнюю программу.

Помимо функций стандартной библиотеки PHP обладает дает механизм Файберов [6]. Файберы в языке программирования PHP – это легковесные потоки выполнения, которые могут быть созданы и управляемы непосредственно внутри приложения, без использования операционной системы.

Основные преимущества Файберов:

1. Низкий уровень накладных расходов – Файберы создаются и управляются на уровне приложения, что позволяет избежать накладных расходов, связанных с созданием и управлением потоками операционной системы.
2. Удобство управления – Файберы предоставляют более простой и удобный способ управления асинхронным выполнением программного кода приложения по сравнению с потоками, так как они работают в рамках единого процесса.
3. Избежание проблем с синхронизацией – поскольку Файберы выполняются в рамках одного процесса, избегается необходимость синхронизации доступа к общим ресурсам, что может упростить разработку и снизить вероятность возникновения проблем с многопоточностью.

Однако Файберы также обладают рядом следующих недостатков:

1. Отсутствие параллелизма на уровне процессора – поскольку Файберы выполняются в рамках одного процесса, они не могут эффективно использовать многопроцессорные системы для параллельной обработки данных.
2. Возможные проблемы с производительностью – несмотря на то, что Файберы обладают низкими накладными расходами, использование слишком большого количества Файберов может привести к снижению производительности из-за переключения контекста основного процесса выполнения.

В целом, Файберы в РНР представляют собой удобный инструмент для организации асинхронного выполнения кода в рамках одного процесса, но требуют внимательного использования для избежания возможных проблем с производительностью и ограничений на уровне языка.

1.3.2 Использование сторонних библиотек

В дополнение к богатой стандартной библиотеке язык программирования PHP обладает большим количеством сторонних библиотек, которые позволяют добавлять в приложения, разработанные на этом языке программирования, различную функциональность. Многие из этих библиотек создаются большими командами, являются востребованными и поддерживаются в актуальном состоянии.

Одной из таких является библиотека асинхронного программирования AMPHP [7]. Она позволяет писать код, который может эффективно обрабатывать множество задач одновременно, не блокируя основной поток выполнения. Библиотека даёт возможность создавать легковесные асинхронные потоки выполнения, которыми можно управлять. Для этого разработан ряд компонентов по работе с асинхронными операциями, такими как сетевые запросы, ввод/вывод операций и работе с событиями. AMPHP может быть использована для написания высокопроизводительных веб-серверов, обработки больших объемов данных, или в других сценариях, требующих асинхронности.

AMPHP обладает рядом преимуществ:

1. Улучшение производительности приложения – использование асинхронного программирования способствует улучшению производительности веб-приложений.
2. Улучшение масштабируемости – приложения, использующие AMPHP, проще масштабировать, так как появляется возможность асинхронного выполнения длительных операций.
3. Широкий набор инструментов – библиотека предоставляет широкий набор инструментов для асинхронного программирования.

Однако также библиотека обладает рядом недостатков:

1. Увеличение сложности разработки – возрастает сложность разработки веб-приложений и веб-сайтов с использованием AMPHP, потому что программистам приходится использовать иной подход, отличающийся от синхронного программирования.

2. Увеличение сложности отладки приложений и сайтов.

1.3.3 Использование архитектурного подхода «Очередь-обработчик»

Архитектурный подход "Очередь-обработчик" – это популярный метод проектирования веб-приложений, который позволяет улучшить производительность, надежность и масштабируемость системы путем добавления новых сервисов в инфраструктуру, которой принадлежит РНР-приложение.

Основная идея этого подхода заключается в том, чтобы разделить выполнение процессов в приложении на две части: первая часть отвечает за прием и сохранение сообщений в очередь, в то время как вторая часть занимается их обработкой. Таким образом, приложение может обрабатывать большое количество HTTP-запросов или запускать множество длительных внутренних процессов, а получение результата выполнения поручается другим сервисам, называемых обработчиками.

В роли серверов очередей, называемых иначе шинами данных, выступают брокеры сообщений. Брокер сообщений [8] – это программное обеспечение, которое позволяет веб-приложениям, системам и службам взаимодействовать друг с другом и обмениваться информацией. Брокер сообщений предоставляет возможность различным сервисам «общаться» друг с другом напрямую, даже если они написаны на разных языках программирования или реализованы на разных платформах.

Брокеры сообщений могут проверять, хранить, маршрутизировать и доставлять сообщения. Они служат посредниками между различными приложениями, позволяя отправлять сообщения, не зная, где находятся получатели, активны они или нет и сколько их. Это облегчает разделение процессов и служб внутри программных систем.

Для обеспечения надежного хранения сообщений и гарантированной доставки, брокеры сообщений часто полагаются на подструктуру или компонент, называемый очередью сообщений. Очередь хранит и упорядочивает

поступающие сообщения. В очереди сообщения хранятся в том же порядке, в котором они были отправлены, и остаются там до тех пор, пока не будет подтверждено получение.

Одним из самых популярных брокеров сообщений является RabbitMQ [9] – это популярный брокер сообщений, который используется для обмена данными между различными компонентами программной системы. Он реализует протокол AMQP, который определяет формат и поведение сообщений, обменников, очередей и других элементов шины данных. Обменником называется компонент RabbitMQ, который принимает сообщения отправителя и занимается их маршрутизацией по очередям в зависимости от настроек обменника. Очереди и обменники связываются между собой при их первоначальной настройке.

В роли обработчиков очередей с сообщениями выступают РНР-воркеры. РНР-воркер – это отдельный фоновый процесс, который выполняется другим РНР-приложением с помощью скрипта, отслеживающего определенную очередь в брокере, забирающего из нее сообщения и выполняющего программный код по обработке сообщения.

Для корректной работы РНР-воркера требуется установить необходимые расширения на сервер, где планируется запуск воркера. Примером устанавливаемого расширения на сервер с РНР-воркером может служить расширение AMQP, необходимое для взаимодействия с RabbitMQ. Далее требуется настроить подключение к брокеру сообщений в РНР-приложении. Для этого обычно используются конфигурационные файлы, в которых указываются параметры подключения к брокеру: хост, порт, логин, пароль и другие. Важно отметить, что описанная ранее настройка необходима не только РНР-воркеру, но и РНР-приложению, которое отправляет сообщения в очередь брокера.

Веб-приложения, реализованные с использованием архитектурного подхода «Очередь-обработчик», обладают следующими преимуществами:

1. Увеличение производительности – благодаря разделению процессов приема и обработки HTTP-запросов система может более

эффективно использовать свои ресурсы, уменьшая нагрузку на приложение и повышая его отзывчивость.

2. Обеспечение надежности – использование очереди позволяет избежать потери данных в случае временных сбоев или перегрузок системы, так как сообщения сохраняются и обрабатываются по мере возможности.
3. Улучшение масштабируемости – архитектура "очередь-обработчик" облегчает масштабирование системы за счет возможности добавления новых обработчиков и использования распределенных очередей для балансировки нагрузки.

Однако также существуют недостатки:

1. Использование дополнительных серверных ресурсов – требуются дополнительные сервера для использования брокера сообщений и РНР-воркеров.
2. Возникновение несогласованности данных – в случаях, когда сообщения в очереди зависят друг от друга или требуют последовательного выполнения.

1.4 Способы отправки электронных сообщений в РНР-приложениях

1.4.1 Использование функций стандартной библиотеки РНР

Стандартная библиотека РНР предоставляет функцию `mail()` [10] для отправки электронных сообщений. Функция возвращает `true`, когда сообщение принято для передачи. Иначе возвращается `false`. То, что сообщение принято для передачи, не означает, что оно будет доставлено. Аргументы функции `mail()` описаны в таблице 2.

Таблица 2 – Аргументы функции mail ()

Тип аргумента	Название аргумента	Описание	Пример
string	\$to	Почта получателя	'me@mail.ru'
string	\$subject	Тема письма	'Тема письма'
string	\$message	Тело письма	'Привет, мир!'
array string	\$additional_headers	Дополнительные заголовки письма	'From: me@mail.ru\r\nReply-To: you@mail.ru'
string	\$additional_params	Дополнительные флаги при отправке письма	'-fme@mail.ru'

Главными достоинствами функции mail () являются простота использования и отсутствие необходимости в подключении сторонних библиотек в PHP-приложение.

Основные недостатки:

1. Отсутствие возможности прикладывать вложения к письму. Например, файлы формата .pdf.
2. По умолчанию длина тела сообщения ограничена 70 символами.
3. Отсутствие возможности контроля доставки письма.

1.4.2 Использование сторонних библиотек

Для более гибкой настройки электронных писем и контроля их отправки часто рекомендуется использовать специальные PHP-библиотеки, такие как SwiftMailer [11]. SwiftMailer представляет собой мощную библиотеку для отправки электронных писем, которая обладает широкими функциональными возможностями и обеспечивает высокую гибкость и надежность.

SwiftMailer позволяет легко настраивать параметры отправки писем, такие как адрес отправителя, адрес получателя, тема письма, текст сообщения и даже вложения. Библиотека обеспечивает удобный интерфейс для работы с различными протоколами, например, SMTP. Также библиотека различные виды шифрования, положительно сказывается на безопасности веб-

приложения, использующих библиотеку. Кроме того, SwiftMailer обладает хорошей документацией, разнообразными примерами использования и активно поддерживается сообществом разработчиков. Это делает библиотеку удобным и надежным инструментом для отправки электронных писем на РНР.

Таким образом, использование библиотеки SwiftMailer для отправки электронных писем на РНР обеспечивает высокую гибкость, простоту использования и надежность, что делает этот инструмент предпочтительным выбором для реализации функционала отправки электронных писем в РНР-приложениях.

1.5 Обзор аналогов

1.5.1 Принцип отбора аналогов

Перед разрабатываемым методом стоит задача улучшить производительность отправки электронных писем по сравнению с традиционной синхронной отправкой писем. Также необходимо, чтобы разрабатываемый метод было возможно внедрить в уже существующее РНР-приложение, была возможность контроля выполнения отправки писем и обработки ошибок в случае их возникновения. Дополнительно необходимо учесть простоту масштабирования системы в случае увеличения нагрузки, возможность гибкой настройки системы и доставляемых электронных писем.

Поиск аналогов производился среди научных статей в электронной библиотеке eLibrary.ru, а также при помощи Google Scholar и поисковой системы Google.

1.5.2 Отобранные аналоги

1. Разработка модуля асинхронной отправки электронных писем с помощью функций стандартной библиотеки `exes()` и `mail()`.

Благодаря большому функционалу, который предоставляет стандартная библиотека языка программирования РНР, возможно реализовать асинхронный метод отправки электронных писем и использованием функций `exes()` и

mail (). В таком случае создается отдельный скрипт вне PHP-приложения, который будет заниматься отправкой электронных писем с помощью функции mail (), запуск PHP-приложением в основном процессе выполнения с помощью функции exec ().

2. Разработка модуля асинхронной отправки электронных писем с помощью сторонних библиотек AMPHP и SwiftMailer.

Для асинхронной отправки электронных писем могут быть использованы сторонние библиотеки AMPHP и SwiftMailer. Возможно разработать отдельный модуль для отправки электронных писем с помощью библиотеки асинхронного программирования AMPHP и библиотеки SwiftMailer для кастомизации электронных писем и отслеживания процесса их отправки, а затем внедрить этот модуль в существующее PHP-приложение.

3. Разработка модуля асинхронной отправки электронных писем с помощью механизма Файберов и библиотеки SwiftMailer.

Для асинхронной отправки электронных писем могут быть использованы механизм Файберов и библиотека SwiftMailer. Возможно разработать отдельный модуль для отправки электронных писем с помощью механизма Файберов и библиотеки SwiftMailer.

1.5.3 Критерии сравнения аналогов

1. Сложность интеграции метода в PHP-приложение

Данный критерий является важным, потому что предполагает оценку уровня сложности интеграции функции, технологии или библиотеки, с помощью которой будет реализован метод, в уже существующий программный продукт. Сложность внедрения напрямую зависит потенциального количества задач и объема этих задач. Например, может оказаться, что для внедрения какой-либо библиотеки в веб-приложение потребуется провести массовый рефакторинг уже существующей кодовой базы приложения. Либо может потребоваться создание дополнительного сервера, где будет установлен, например, брокер сообщений. Такие задачи увеличивают сложность интеграции

функции, библиотеки или технологии. Также стоит отметить, что технологии, при помощи которых разработан внедряемый метод должны быть совместимы с той версией PHP, которую поддерживает приложение.

2. Производительность метода

Данный критерий является одним из ключевых критериев для оценки методов по отправке электронных сообщений. Улучшение производительности метода подразумевает сокращение времени ожидания ответа на запрос к PHP-приложению. Высокой считается производительность, когда длительные операции подобные отправке электронных сообщений, минимально влияют на время, за которое веб-приложение обрабатывает запрос пользователя. Низкой считается производительность в случае, когда длительные операции напрямую влияют на время ожидания ответа пользователем от программного продукта, разработанного на языке программирования PHP.

3. Масштабируемость метода

Данный критерий является важным, потому что в случаях увеличения нагрузки на приложение необходима возможность большего количества электронных писем путем прогнозируемого добавления ресурсов сервера или увеличения количества серверов.

4. Надежность метода

Данный критерий является важным, потому что предполагает оценку надежности разрабатываемого метода асинхронной отправки электронных писем. Метод с высокой надежностью подразумевает наличие возможности убедиться в том, что необходимые электронные письма были доставлены, иначе позволяет обработать ошибки, возникшие во время отправки и отправить письмо повторно.

1.5.4 Сравнение аналогов

В таблице 3 представлено сравнение аналогов по описанным ранее критериям.

Таблица 3 – Сводная таблица сравнения аналогов

Аналог/Критерий	Сложность интеграции метода	Производительность метода	Масштабируемость метода	Надежность метода
Модуль с использованием функции стандартной библиотеки	Низкая	Низкая	Низкая	Низкая
Модуль с использованием AMPHP и SwiftMailer	Высокая	Умеренная	Низкая	Умеренная
Модуль с использованием Файберов и SwiftMailer	Умеренная	Умеренная	Умеренная	Низкая

1.5.5 Выводы по итогам сравнения

Из сравнения аналогов можно сделать вывод, что в большей мере поставленную задачу решают аналоги, которые включают в себя использование сторонней библиотеки SwiftMailer и механизма Файберов стандартной библиотеки PHP или сторонней библиотеки AMPHP. Однако слабой стороной данных аналогов являются высокая сложность интеграции библиотеки или механизма в уже существующее PHP-приложение, так как потребуются произвести массовый рефакторинг существующей кодовой базы. Дополнительно стоит учитывать, что как механизм Файберов, так и библиотека AMPHP требуют поддержки определенной версии языка программирования PHP, что может встать преградой при использовании данных инструментов. А также на оба аналога обладают низкой масштабируемостью, потому что не позволяют прогнозируемо управлять ресурсами сервера при увеличении нагрузки.

Выявленные сильные и слабые стороны аналогов позволяют сформулировать требования к собственному методу, которые рассматриваются в разделе «Перечень требований к решению».

2 ФОРМУЛИРОВКА ТРЕБОВАНИЙ К РЕШЕНИЮ И ПОСТАНОВКА ЗАДАЧИ

2.1 Постановка задачи

Необходимо разработать асинхронный метод по отправке электронных писем в РНР-приложениях и провести его исследование.

2.2 Требования к решению

Разрабатываемый метод асинхронной отправки электронных писем в РНР-приложении должен отвечать следующим требованиям:

- Метод должен отправлять электронные письма асинхронно – данное требование подразумевает, что основной поток выполнения программы, то есть обработка запроса пользователя к веб-приложению, не должен блокироваться операцией отправки электронного письма.
- Необходимо обеспечить умеренную сложность интеграции метода в РНР-приложение – данное требование подразумевает, что разрабатываемый метод должен придерживаться уже используемых в РНР-приложении возможностей стандартной библиотеки языка, сторонних библиотек или технологий, а также придерживаться существующей архитектуры программного кода приложения.
- Необходимо обеспечить высокую производительность метода – данное требование подразумевает сокращение времени ожидания ответа пользователем от РНР-приложения.
- Необходимо обеспечить высокую масштабируемость метода – данное требование подразумевает подконтрольное увеличение ресурсов сервера или количества серверов при возрастании нагрузки на приложение.
- Необходимо обеспечить высокую надежность метода – данное требование подразумевает возможность проверить успешность доставки электронного письма и возможность повторной отправки письма в случае возникновения ошибки отправки.

- Необходимо обеспечить гибкую настройку отправляемых электронных писем – данное требование подразумевает возможность настройки отправляемых электронных писем. Например, возможность отправить содержание письма в виде сформированной HTML-разметки или возможность добавления файлов разных форматов к письму: .pdf, .png и т.п.

Дополнительно были сформированы программные требования к методу в соответствии с технологиями, на которых реализовано PHP-приложение Colibri.travel, в которое планируется внедрять разрабатываемый метод.

- Программные требования:
 - Операционная система: Linux Ubuntu 20
 - Платформа: PHP 8.0.26
 - Фреймворк Laravel 9
 - База данных: PostgreSQL 13.0
 - Брокер сообщений: RabbitMQ 3.12

2.3 Обоснование выбора метода решения

Для реализации метода асинхронной отправки электронных писем было принято решение использовать архитектурный подход «Очередь-обработчик», стороннюю библиотеку SwiftMailer для гибкой настройки отправляемых электронных писем.

Архитектурный подход «Очередь-обработчик» позволит выполнять отправку электронных писем асинхронно, благодаря тому что PHP-приложение, которому в ходе обработки пользовательского запроса необходимо отправить электронное письмо, будет помещать сообщение с информацией о письме в очередь, а непосредственной отправкой электронного письма будет заниматься обработчик очереди. Одним из важных преимуществ данного подхода является прогнозируемое потребление ресурсов серверов, так как для каждого обработчика выделяется фиксированное количество ресурсов для его работы.

3 АРХИТЕКТУРА ПРОГРАММНОЙ РЕАЛИЗАЦИИ

3.1 Используемые технологии

Разработка асинхронного метода по отправке электронных писем в РНР-приложениях выполнена с использованием высокоуровневого языка программирования РНР. В качестве надстройки над РНР будет использован Laravel [12]. Laravel – это веб-фреймворк для разработки приложений на РНР, который обеспечивает простой и эффективный подход для работы с базами данных и брокерами сообщений, маршрутизации HTTP-запросов, авторизации пользователей и других функций, необходимых при разработке веб-приложений и модулей для них. В качестве СУБД была выбрана PostgreSQL, а в качестве брокера сообщений был выбран RabbitMQ.

Дополнительно при разработке использовались следующие библиотеки:

- Php-amqp-lib [13] – это популярная библиотека, которая предоставляет возможность взаимодействия приложения, разработанного на РНР, с брокером сообщений RabbitMQ по протоколу AMQP. Она предоставляет инструменты для отправки и получения сообщений через очереди, создание очередей и управлением ими.
- SwiftMailer – представляет собой мощную РНР-библиотеку для отправки электронных писем. Библиотека обладает широкими функциональными возможностями и обеспечивает высокую гибкость и надежность: узнавать получателей, которым не удалось доставить электронное письмо, добавлять вложения в виде файлов разных форматах к письму.

3.2 Общая схема работы метода

Общая схема работы метода следующая:

1. Пользователь отправляет запрос к РНР-приложению, в рамках обрабатываемого запроса необходимо отправить электронное письмо.

2. Во время обработки пользовательского запроса приложение формирует электронное письмо и сохраняет его в соответствующую таблицу в базе данных.
3. Вместо непосредственной отправки электронного письма на почту, приложение отправляет уникальный идентификатор письма в подготовленную очередь брокера сообщений.
4. Приложение формирует и отправляет ответ пользователю.
5. Обработчик сообщений получает уникальный идентификатор электронного письма из очереди.
6. Обработчик сообщений находит в базе данных электронное письмо по уникальному идентификатору.
7. Обработчик сообщений отправляет электронное письмо на почту.

На рисунке 1 представлена общая схема работы метода.

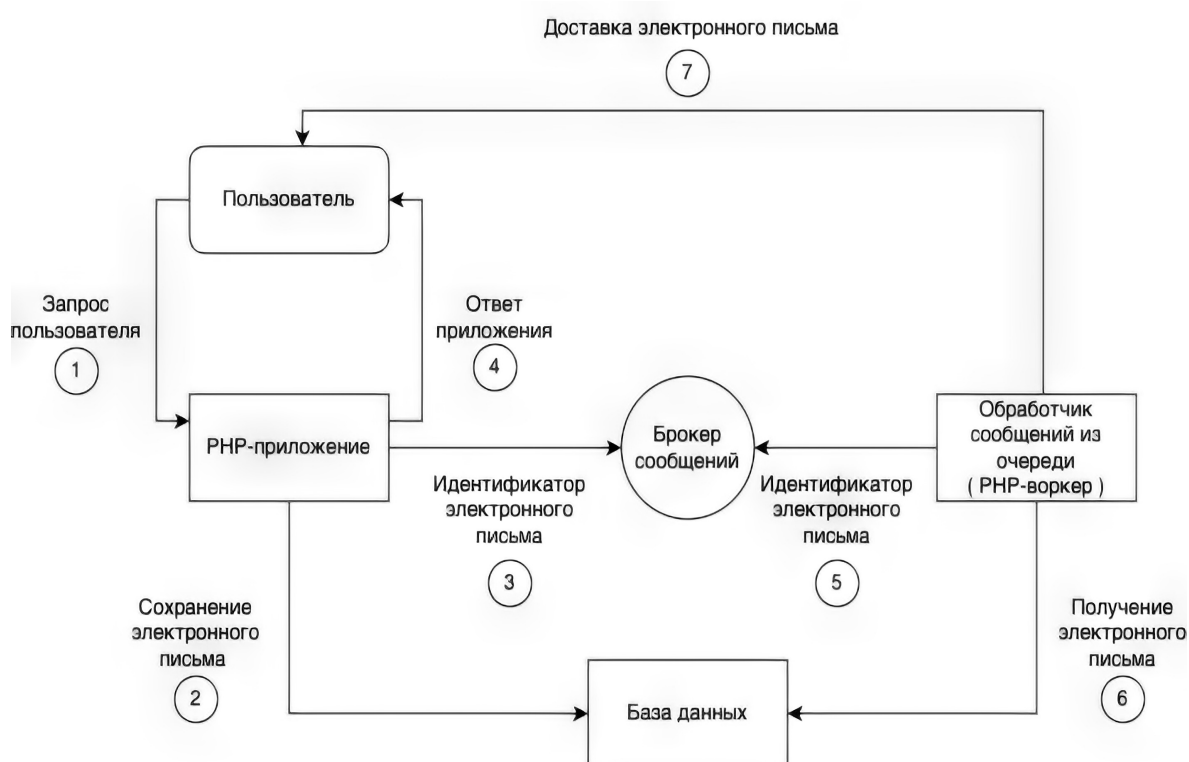


Рисунок 1 – общая схема работы метода

Важно отметить, что обработчик сообщений из очереди не дожидается ответа от РНР-приложения на пользовательский запрос, обработчик непрерывно следит за поступающими в очередь сообщениями и обрабатывает их по мере поступления.

3.3 Проектирование новой таблицы в существующей базе данных

3.3.1 Обоснование проектирования таблицы

Для эффективного и надежного взаимодействия РНР-приложения, брокера сообщений и РНР-воркера было принято решение спроектировать новую таблицу в уже существующей базе данных. Эффективность взаимодействия между РНР-приложением, брокером сообщений и РНР-воркером посредством использования базы данных, как промежуточного звена, хранящего полную информацию об электронном письме, заключается в том, что нет необходимости пересылать полную информацию о письме по сети, используя различные сетевые протоколы. РНР-приложению достаточно сохранить сформированное письмо в базу данных и отправить уникальный идентификатор письма в подготовленную очередь брокера сообщений, а РНР-воркеру найти электронное письмо в соответствующей таблице базы данных по полученному уникальному идентификатору из очереди в брокере сообщений. Надежность такого подхода заключается в том, что в случае отказа РНР-воркера во время обработки сообщения из очереди или отказа брокера сообщений, полная информация об электронном письме не будет утеряна, и будет возможность найти письмо в базе данных и отправить его повторно.

3.3.2 Определение этапов проектирования таблицы

При проектировании новой таблицы в существующей реляционной базе данных логично выделить следующие этапы:

- Определение требований к таблице – на этом этапе определяются цели и задачи хранения данных. Необходимо понять, какие данные будут храниться в таблице и набор операций, выполняемых с этими данными.
- Определение сущности и атрибутов, хранимых в таблице – на основе собранных требований определяется сущность, которая будет представлена в таблице, а также атрибуты этой сущности.

- Определение ключевого атрибута сущности – необходимо определить ключевой атрибут сущности, который будет уникально и однозначно идентифицировать каждую сущность.
- Определение полей таблицы, их типов данных и ограничений – необходимо описать поля проектируемой таблицы и для каждого определить хранимый тип данных, например, целое число или строка. Также необходимо установить ограничения на поля таблицы, например, ограничение уникальности.
- Создание таблицы – последний этап при проектировании новой таблицы. На данном этапе предполагается создание таблицы, используя выбранную СУБД и язык запросов SQL.

3.3.3 Проектирование таблицы

Определение требований к таблице

Таблица `email_messages` предполагает сохранение всех данных, относящихся к электронному письму: электронных адресов отправителя и получателей, темы и тела электронного письма, дату создания письма, ссылок на файлы вложений к письму. Основной функцией проектируемой таблицы является хранение полной информации об электронном письме для увеличения надёжности проектируемого метода по асинхронной отправке электронных писем, а также уменьшение нагрузки на сеть во время передачи информации об электронном письме в брокер сообщений. Предполагается следующий набор операций по работе с данными из проектируемой таблицы:

- Вставка новых записей
- Обновление существующих записей
- Поиск записей

Определение сущности и атрибутов, хранимых в таблице

После определения функций таблицы следует определить хранимую сущность. Сущность в теории баз данных — это класс однотипных моделей,

информация о которых требуется для функционирования системы. В моделях ER сущность изображается прямоугольником, внутри которого размещается название сущности. Сущность обязательно состоит из атрибутов. Атрибут – это свойство сущности, которое описывает конкретную характеристику.

Для таблицы email_messages была выделена сущность: «Электронное письмо» и следующий набор атрибутов: уникальный идентификатор, адрес отправителя, адреса получателей, адреса получателей копии письма, тема письма, тело письма, дата создания письма, дата отправки письма, признак успешности отправки письма, ссылки на файлы вложений к письму.

После определения сущности, её атрибутов и связей с другими сущностями, хранимыми в таблицах существующей базы данных следует построение ER-модели проектируемой таблицы.

На рисунке 2 представлена ER-модель проектируемой таблицы.



Рисунок 2 – ER-модель сущности «Электронное письмо»

Определение ключевого атрибута сущности

Ключевым атрибутом сущности «Электронное письмо» будет «уникальный идентификатор», каждая сущность, ассоциируемая с записью в таблице email_messages, будет обладать своим уникальным идентификатором, что позволит однозначно определить искомую запись.

Определение полей таблицы, их типов данных и ограничений

В таблице 4 представлены поля таблицы email_messages, их типы данных и необходимые ограничения в соответствии с обозначенной ранее сущностью «Электронное письмо» и её атрибутами.

Возможные виды ограничений, представленных в таблице 4:

- PK – PRIMARY KEY, ограничение первичного ключа указывает, что поле или группа полей могут использоваться в качестве уникального идентификатора строк в таблице.
- NN – NOT NULL, данное ограничение указывает, что поле не может принимать нулевое значение.
- SR – SERIAL, псевдо тип данных в PostgreSQL, который используется для поля или группы полей, являющихся уникальным идентификатором в таблице. Позволяет задать последовательность уникальных целых чисел с автоматическим инкрементом при вставке новой записи в таблицу.

Таблица 4 – Определение полей таблицы email_messages

Название поля	Описание	Тип поля	Ограничения
email_message_id	Уникальный идентификатор	SERIAL	SR, PK
sender_email	Адрес отправителя	VARCHAR(255)	NN
recipients	Адреса получателей	JSON	NN
copies	Адреса получателей в копии	JSON	-
created_at	Дата создания	TIMESTAMP	NN
sent_at	Дата отправки	TIMESTAMP	-
subject	Тема письма	TEXT	NN
body	Тело письма	TEXT	NN
is_sent	Признак успешной отправки	BOOLEAN	-
attachments	Ссылки на файлы вложений	JSON	-

Создание таблицы

Финальным этапом при проектировании таблицы email_messages будет создание таблицы, используя СУБД PostgreSQL и язык запросов SQL.

Таблица email_messages реализована с помощью SQL-скрипта, представленного на рисунке 3:

```
CREATE TABLE `email_messages` (  
  email_message_id SERIAL,  
  sender_email VARCHAR(255) NOT NULL,  
  recipients JSON NOT NULL,  
  copies JSON,  
  subject TEXT NOT NULL,  
  body TEXT NOT NULL,  
  is_sent BOOLEAN,  
  attachments JSON,  
  created_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,  
  sent_at TIMESTAMP WITHOUT TIME ZONE,  
  PRIMARY KEY(email_message_id)  
);
```

Рисунок 3 – SQL-скрипт создания таблицы email_messages

3.4 Конфигурационные файлы

3.4.1 Конфигурационный файл queue.php

Для обеспечения возможности подключения PHP-приложения к брокеру сообщений RabbitMQ был разработан конфигурационный файл queue.php.

Данный файл содержит ассоциативный массив данных для подключения к различным шинам данных. Ключом в массиве является название шины данных, а значением – массив, содержащий данные для подключения.

На рисунке 4 представлен пример конфигурационного файла queue.php.

```
<?php  
  
return [  
  'rabbitmq' => [  
    'host' => 'hostName',  
    'port' => 'port',  
    'user' => 'userName',  
    'password' => 'password',  
  ],  
];
```

Рисунок 4 – пример конфигурационного файла queue.php

В таблице 5 представлено описание полей секции rabbitmq.

Таблица 5 – описание полей секции rabbitmq файла queue.php

Название поля	Тип поля	Описание поля
host	string	Адрес сервера, на котором запущен брокер сообщений
port	string	Порт сервера, который прослушивается брокером сообщений
user	string	Логин пользователя для подключения к брокеру сообщений
password	string	Пароль пользователя для подключения к брокеру сообщений

3.4.2 Конфигурационный файл eventbus.php

Для обеспечения гибкости настройки внутренней конфигурации брокера сообщений напрямую из РНР-приложения был разработан конфигурационный файл eventbus.php.

Конфигурационный файл содержит основную секцию services представляющую собой ассоциативный массив, где каждому сервису, работу которого необходимо реализовать через шину данных, сопоставлен двумерный массив настроек шины данных.

Каждый элемент секции services предполагает следующий набор вложенных секций:

- exchangeParams – массив параметров для настройки обменника в брокере сообщений.
- queueParams – массив параметров для настройки очереди в брокере сообщений.
- messageParams – массив параметров для настройки сообщения, отправляемого в очередь брокера сообщений.
- handlerParams – массив параметров для настройки обработчика сообщений из очереди в брокере.

На рисунке 5 представлен пример конфигурационного файла eventbus.php.

```
<?php
declare(strict_types=1);

return [
    'services' => [
        'email_message_sending' => [
            'exchangeParams' => [
                'name' => 'email_message_sending',
                'type' => 'direct',
                'passive' => false,
                'durable' => false,
                'autoDelete' => true,
            ],
            'queueParams' => [
                'name' => 'email_message_sending',
                'passive' => false,
                'durable' => false,
                'exclusive' => false,
                'autoDelete' => false,
            ],
            'messageParams' => [
                'deliveryMode' => 2,
                'contentEncoding' => 'UTF8',
            ],
            'handlerParams' => [
                'prefetchCount' => 1,
                'noAck' => false,
                'consumerExclusive' => false,
            ],
        ],
    ],
];
```

Рисунок 5 – пример конфигурационного файла eventbus.php

В таблице 6 представлено описание полей секции exchangeParams.

Таблица 6 – описание полей секции exchangeParams файла eventbus.php

Название поля	Тип поля	Описание поля
name	string	Название обменника в брокере сообщений
type	string	Тип обменника в брокере сообщений
passive	boolean	Флаг, определяющий поведение брокера сообщений в случае запроса обменника по имени

Продолжение таблицы 6

durable	boolean	Флаг, определяющий поведение обменника при перезагрузке брокера сообщений
autoDelete	boolean	Флаг, определяющий поведение обменника, когда отсутствуют связанные с ним очереди

В соответствии с указанными настройками в брокере сообщений будет создан обменник `email_message_sending`, обладающий типом `direct` – поступающие сообщения будут отправляться в очереди с ключом, соответствующим названию обменника. При запросе по имени несуществующего обменника, будет возвращаться ошибка о том, что такого обменника не существует. Обменник будет настроен таким образом, что после перезагрузки брокера сообщений он не сохранится, если в какой-либо момент времени в обменнике будут отсутствовать очереди, то он удалится. Такая настройка обменника обусловлена тем, что в случае перезагрузки брокера сообщений нет необходимости сохранять информацию об электронных письмах, потому что она уже сохранена в базе данных, поэтому серверу, где расположен брокер сообщений можно выделить меньше ресурсов или создать большее количество других обменников и очередей.

В таблице 7 представлено описание полей секции `queueParams`.

Таблица 7 – описание полей секции `queueParams` файла `eventbus.php`

Название поля	Тип поля	Описание поля
name	string	Название очереди в брокере сообщений
passive	boolean	Флаг, определяющий поведение брокера сообщений в случае запроса очереди по имени

Продолжение таблицы 7

durable	boolean	Флаг, определяющий поведение очереди при перезагрузке брокера сообщений
autoDelete	boolean	Флаг, определяющий поведение очереди, когда в ней отсутствуют сообщения
exclusive	boolean	Флаг, определяющий эксклюзивность очереди

В соответствии с указанными настройками в обменнике `email_message_sending` брокера сообщений будет создана очередь `email_message_sending`. При запросе по имени несуществующей очереди, будет возвращаться ошибка о том, что такой очереди не существует. При перезагрузке брокера сообщений очередь и сообщения, расположенные в ней, не будут сохраняться. При отсутствии сообщений в очереди очередь не будет удалена, и очередь сможет иметь неограниченное количество обработчиков. Такие настройки обусловлены тем, что в случае перезагрузки сообщений нет необходимости сохранять информацию об электронных письмах, так как эта информация уже находится в базе данных. А также в случае увеличения нагрузки на очередь `email_message_sending` будет возможность увеличить количество обработчиков очереди.

В таблице 8 представлено описание полей секции `messageParams`.

Таблица 8 – описание полей секции `exchangeParams` файла `eventbus.php`

Название поля	Тип поля	Описание поля
<code>deliveryMode</code>	<code>integer</code>	Отвечает за сохранность сообщения в случае перезагрузки брокера сообщений
<code>contentEncoding</code>	<code>string</code>	Тип кодировки тела сообщения, отправляемого в очередь

Параметр `deliveryMode` принимает значения из множества $\{1, 2\}$. В случае `durable` очереди при перезагрузке брокера значение 1 обозначает потерю

сообщения в очереди, 2 – сохранение сообщения. Поэтому сообщения, отправляемые в очередь `email_message_sending`, не будут сохраняться при перезагрузке брокера сообщений и будут кодироваться в формате UTF-8.

В таблице 9 представлено описание полей секции `handlerParams`.

Таблица 9 – описание полей секции `handlerParams` файла `eventbus.php`

Название поля	Тип поля	Описание поля
<code>prefetchCount</code>	<code>integer</code>	Число сообщений, которое готов принять обработчик в единичный момент времени
<code>noAck</code>	<code>boolean</code>	Флаг, обозначающий обязанность обработчика оповестить брокер после обработки сообщения
<code>consumerExclusive</code>	<code>boolean</code>	Флаг, определяющий, что у очереди будет всего один обработчик

В соответствии с указанными настройками число сообщений, которое готов принять обработчик в единичный момент времени равно 1, после обработки сообщения обработчик будет оповещать брокер сообщений о статусе обработки, и будет возможность задать несколько обработчиков очереди.

3.4.3 Конфигурационный файл `worker.php`

Для определения обработчиков, запускаемых в RHP-воркерах был разработан конфигурационный файл `worker.php`.

Данный файл обладает основной секцией `workers`, которая представляет собой ассоциативный массив. Ключом в этом массиве выступает название обрабатываемого процесса, а значением является массив, содержащий класс обработчика процесса.

На рисунке 6 представлен пример конфигурационного файла `worker.php`.

```

<?php
declare(strict_types=1);

return [
    'workers' => [
        'email_message_sending' => 'handlerClassName',
    ],
];

```

Рисунок 6 – пример конфигурационного файла worker.php

3.5 Структура программной реализации

3.5.1 Реализованные классы

При выполнении работы были реализованы следующие программные классы и доступные в них методы.

ConfigService

Данный класс служит для получения данных из различных конфигурационных из файлов. Например, получение настроек для подключения РНР-приложения к брокеру сообщений или подключение к почтовому серверу.

В таблице 10 представлены методы класса и их краткое описание

Таблица 10 – описание методов класса ConfigService

Сигнатура метода	Описание метода
public function getEventBusConfig(): array	Данный метод возвращает настройки для взаимодействия с шиной данных
public function getWorkersConfig(): array	Данный метод возвращает настройки РНР-воркеров, объявленных в приложении
public function getSmtpConfig(): array	Данный метод возвращает настройки SMTP-почтового сервера, с которым взаимодействует РНР-приложение

EventBusInterface

Данный интерфейс определяет сигнатуры методов по взаимодействию между шинами данных и РНР-приложением. Создание интерфейса предполагает, что в случае, когда потребуется подключить к приложению какую-либо

иную шину данных, достаточно будет создать новый класс, реализующий данный интерфейс, и указать использование новой реализации интерфейса через специальные конфигурационные файлы. В таком случае не потребуется перерабатывать существующую кодовую базу для использования новой шины данных, что значительно повышает гибкость разрабатываемого программного решения.

В таблице 11 представлены методы интерфейса и их краткое описание

Таблица 11 – описание методов интерфейса EventBusInterface

Сигнатура метода	Описание метода
public function publish(\$message): void	Данный метод отвечает за отправку сообщения в шину данных. Метод принимает строковое представление отправляемого сообщения
public function subscribe(\$handler): void	Данный метод отвечает за непрерывное отслеживание сообщений, поступающих в шину данных, и их обработку. Метод принимает класс, реализующих интерфейс обработчика сообщений из шины данных

HandlerInterface

Данный интерфейс определяет сигнатуры методов по обработке сообщений, поступающих из шины данных. Создание интерфейса позволяет определить любое количество обработчиков сообщений из шины данных, где каждый обработчик будет обладать своей уникальной логикой по работе с сообщением, поступившим на обработку.

В таблице 12 представлены методы интерфейса и их краткое описание

Таблица 12 – описание методов интерфейса HandlerInterface

Сигнатура метода	Описание метода
public function handle(\$message): void	Данный метод отвечает за обработку сообщения из шины данных. Метод принимает строковое представление сообщения, которое необходимо обработать

RabbitMQService

Данный класс реализует интерфейс EventBusInterface и обладает методами по взаимодействию PHP-приложения с брокером сообщений RabbitMQ. Класс использует внешнюю библиотеку php-amqp и реализует подключение PHP-приложения к брокеру сообщений, создание и настройку каналов, обменников и очередей в брокере сообщений, публикацию, отслеживание и обработку сообщений.

В таблице 13 представлены методы класса и их краткое описание

Таблица 13 – описание методов класса RabbitMQService

Сигнатура метода	Описание метода
public function __construct(\$connectionParams, \$exchangeParams, \$queueParams, \$messageParams, \$handlerParams)	Конструктор класса. Инициализирует объект класса настройками из файла eventbus.php
private function getConnection(): AbstractConnection	Данный метод создает подключение к брокеру сообщений
private function getChannel(): AMQPChannel	Данный метод возвращает канал брокера сообщений, к которому выполнено подключение

Продолжение таблицы 13

public function handleCallback(\$AMQPMessage): void	Данный метод вызывает обработку сообщения из очереди
private function close(): void	Данный метод закрывает подключение к брокеру сообщений
private function prepareExchangeQueue(): array	Данный метод создает или подключается к уже созданному обменнику и очереди в брокере сообщений
public function publish(\$message): void	Данный метод публикует сообщение в подготовленную очередь брокера сообщений
public function subscribe(\$handler): void	Данный метод инициализирует обработчик очереди в брокере сообщений и вызывает обработку очереди до тех пор, пока не закрыто подключение

AMQPMessageAssembler

Данный класс предназначен для создания экземпляра сообщения AMQPMessage на основании переданных строкового представления тела и настроек сообщения. Данное сообщение предназначено для отправки в очередь брокера сообщений.

В таблице 14 представлены методы класса и их краткое описание

Таблица 14 – описание методов класса AMQPMessageAssembler

Сигнатура метода	Описание метода
public function assemble(\$message, \$messageParams): void	Создание экземпляра сообщения AMQPMessage

EmailMessage

Это класс электронного письма, который обеспечивают взаимодействие с базой данных. Данный класс является представителем моделей во фреймворке Laravel. Модель предоставляет интерфейс для работы с таблицей базы данных и позволяет выполнять различные операции с данными таблицы,

такими как создание, чтение, обновление и удаление записей. Данный класс содержит в себе все поля, перечисленные при проектировании таблицы email_messages: email_message_id, co_email, recipients, copies, created_at, sent_at, subject, body, is_sent, attachments.

EmailMessageSendingTaskHydrator

Данный класс предназначен для преобразования целочисленного уникального идентификатора электронного письма в строковый формат и обратного преобразования. Класс используется для подготовки тела сообщения, которое будет отправлено в брокер сообщений, а также для обратного преобразования тела сообщения из брокера в уникальный идентификатор электронного письма.

В таблице 15 представлены методы класса и их краткое описание

Таблица 15 – описание методов класса EmailMessageSendingTaskHydrator

Сигнатура метода	Описание метода
public function extract(\$emailMessageId): string	Метод предназначен для преобразования целочисленного уникального идентификатора электронного письма в строковый формат
public function hydrate(\$data): int	Метод предназначен для преобразования строкового уникального идентификатора электронного письма в целочисленный формат

AsyncEmailMessageSendingService

Класс предназначен для отправки уникального идентификатора электронного письма в подготовленную очередь брокера сообщений.

В таблице 16 представлены методы класса и их краткое описание

Таблица 16 – описание методов класса AsyncEmailMessageSendingService

Сигнатура метода	Описание метода
public function __construct(\$eventBusInterface, \$emailMessageSendingTaskHydrator)	Конструктор класса. Инициализирует текущий объект классом, реализующим интерфейс для работы с шиной данных и классом, выполняющим подготовку тела сообщения к отправке в брокер сообщений
public function send(\$emailMessage): void	Метод предназначен для подготовки и отправки экземпляра класса электронного письма в очередь брокера сообщений

EmailMessageSendingTaskHandler

Данный класс реализует интерфейс HandlerInterface и является обработчиком тела сообщения, поступившего из очереди в брокере сообщений.

В таблице 17 представлены методы класса и их краткое описание

Таблица 17 – описание методов класса EmailMessageSendingTaskHandler

Сигнатура метода	Описание метода
public function __construct(\$emailMessageSendingTaskHydrator, \$emailMessageSendingTaskService)	Конструктор класса. Инициализирует текущий объект классом для преобразования тела сообщения из очереди в уникальный идентификатор электронного письма и классом, отправляющим электронное письмо по уникальному идентификатору
public function handle(\$message): void	Метод предназначен для обработки сообщения, поступившего из очереди в брокере сообщений

EmailMessageSendingTaskService

Данный класс выполняет поиск письма в базе данных по уникальному идентификатору, и транслирует полученную модель электронного письма EmailMessage в объект класса Swift_Message сторонней PHP-библиотеки

SwiftMailer, а далее выполняет отправку электронного письма на почту, и, в случае успешной отправки, обновляет модель электронного письма датой отправки и признаком успешности отправки электронного письма.

В таблице 18 представлены методы класса и их краткое описание

Таблица 18 – описание методов класса EmailMessageSendingTaskHandler

Сигнатура метода	Описание метода
public function __construct(\$Swift_Mailer, \$swiftEmailMessageAdapter)	Конструктор класса. Инициализирует текущий объект классами для отправки письма
public function send(\$emailMessageId): void	Метод выполняет поиск и отправку электронного письма по его уникальному идентификатору, после успешной отправки модель письма обновляется датой отправки

SwiftEmailMessageAdapter

Данный класс преобразовывает модель электронного письма EmailMessage в объект класса Swift_Message библиотеки по отправке электронных сообщений SwiftMailer.

В таблице 19 представлены методы класса и их краткое описание

Таблица 19 – описание методов класса EmailMessageSendingTaskHandler

Сигнатура метода	Описание метода
public function translateToSwiftMessage(\$emailMessage): Swift_Message	Метод предназначен для преобразования модели электронного письма в объект письма внешней библиотеки

WorkerCommand

Данный класс представляет собой консольный скрипт, который запускается PHP-воркером и начинает какой-либо длительный процесс. Для определения того, какой процесс необходимо запустить, в скрипт передаётся обязательный аргумент {worker}, обозначающий имя запускаемого обработчика, далее считывается конфигурационный файл worker.php, определяется и запускается необходимый обработчик.

AsyncEmailMessageSendingController

Данный класс является API-контроллером, который будет обрабатывать запросы на асинхронную отправку электронных писем, поступающие к PHP-приложению.

В таблице 20 представлены методы класса и их краткое описание

Таблица 20 – описание методов класса AsyncEmailMessageSendingController

Сигнатура метода	Описание метода
public function send(\$request): Response	Метод предназначен для обработки запроса по асинхронной отправке электронного письма

Сам запрос, который необходимо отправить к PHP-приложению для асинхронной отправки электронного письма, представлен в таблице 21

Таблица 21 – описание методов класса AsyncEmailMessageSendingController

Запрос	GET https://127.0.0.1/api/async-email-send
Параметры	• senderEmail – электронная почта отправителя • recipients – массив электронных почт получателей • copies – массив электронных почт в копию отправляемого письма • body – тело электронного письма • subject – тема электронного письма • emailsCount – количество отправляемых писем
Пример	https://127.0.0.1/api/async-email-send?sender_email=forest-chaan@mail.ru&recipients[]=fearless_forest-chaan@mail.ru&copies[]=example@mail.ru&body=тело&subject=тема&emailsCount=1

3.5.2 Взаимодействие классов

Все реализованные классы и их методы их взаимодействия можно представить на UML-диаграмме классов, которая представлена на рисунке 7.

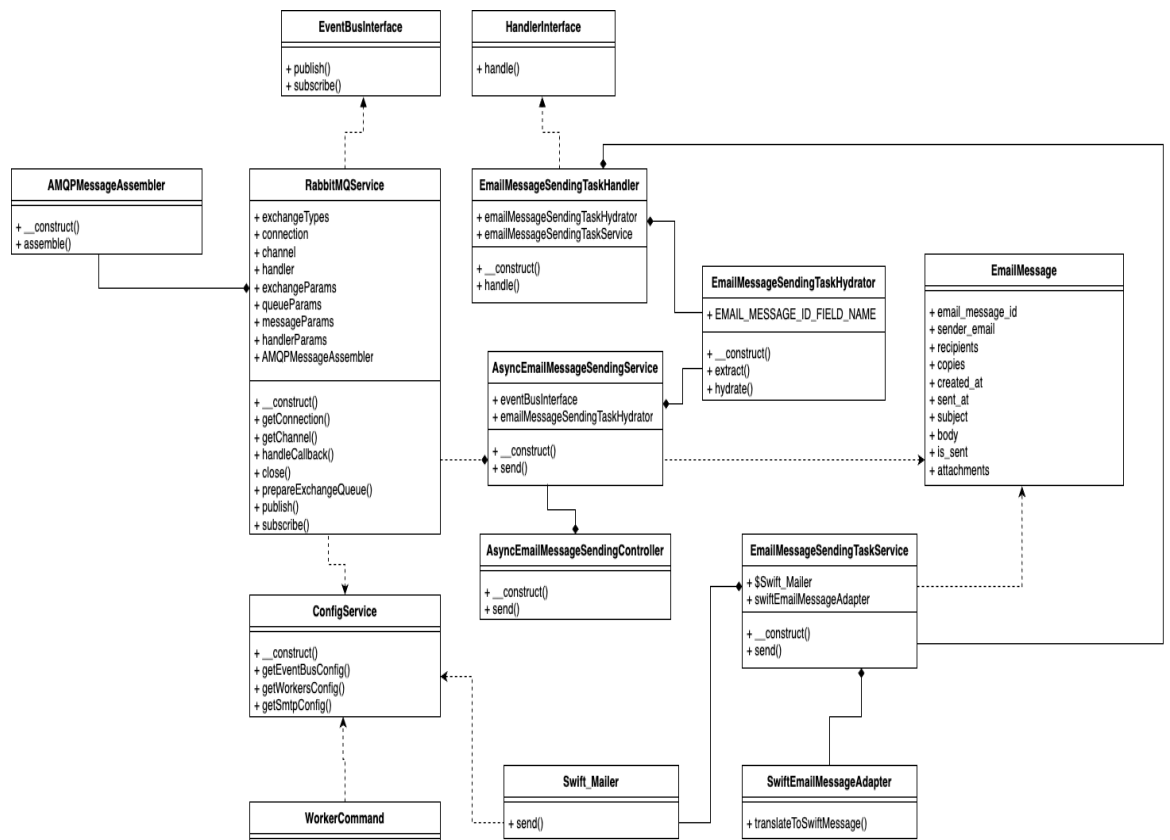


Рисунок 7 – UML-диаграмма классов решения

При получении нового запроса на отправки электронного письма, создается объект класса `AsyncEmailMessageSendingController` проинициализированный объектом класса `AsyncEmailMessageService`. При создании `AsyncEmailMessageService` инициализируются классы `RabbitMQService` и `EmailMessageSendingTaskHydrator`. Во время инициализации класса `RabbitMQService` с помощью метода `getEventBusConfig()` класса `ConfigService` получают необходимые параметры из конфигурационных файлов `queue.php` и `eventbus.php`. Таким образом получается, что объект класса `RabbitMQService` проинициализирован всеми параметрами, необходимыми для работы и настройки шины данных для асинхронной отправки электронных сообщений, и объект класса `AsyncEmailMessageService` агрегирует в себе сконфигурированный объект класса по работе с шиной данных.

Далее создается объект класса `EmailMessage`, который содержит в себе всю необходимую информацию об электронном письме, переданную в запросе, объект класса электронного письма `EmailMessage` передается в метод `send()` класса `AsyncEmailMessageService` и преобразовывается в строковый

формат при помощи метода `extract()` класса `EmailMessageSendingTaskHydrator`, а затем публикуется в очередь брокера сообщений при помощи метода `publish()` класса `RabbitMQService`.

Перед публикацией первого сообщения в брокер производится настройка обменника, очереди и привязка очереди к обменнику при помощи внутренних методов объекта класса `RabbitMQService`.

Далее заранее запущенный и отслеживающий необходимую очередь скрипт `WorkerCommand` в PHP-воркере получает поступившие в очередь сообщения и начинает их обработку с помощью созданного объекта класса обработчика `EmailMessageSendingTaskHandler`.

При обработке полученного текстового сообщения происходит его преобразование в целочисленный формат с помощью метода `hydrate()` и по полученному уникальному идентификатору сообщения находит его в базе данных. При нахождении электронного письма в базе данных получается объект класса `EmailMessage`, который преобразуется в объект класса `Swift_Message` методом `translateToSwiftMessage()` класса `SwiftEmailMessageAdapter`, и отправляется на почту при помощи класса `Swift_Mailer` PHP-библиотеки по работе с электронной почтой `SwiftMailer`.

4 ИССЛЕДОВАНИЕ РАЗРАБОТАННОГО МЕТОДА

4.1 Исследование времени отправки электронных писем

Так как одним из основных требования при разработке метода является его производительность, а именно скорость ответа PHP-приложения на поступающие запросы, в рамках обработки которых необходимо отправить электронное письмо, то необходимо исследовать и сравнить время отправки электронных писем с использованием синхронного и асинхронного методов.

Для исследования скорости обработки запросов с синхронной и асинхронной отправкой электронных писем использовались функция стандартной библиотеки PHP `microtime()`, позволяющая зафиксировать момент времени своего вызова, и был дополнительно разработан класс `SyncEmailMessageSendingController`, который является аналогом API-контроллера `AsyncEmailMessageSendingController`, но для синхронной отправки электронных писем. Для вычисления времени обработки запроса по отправке электронных писем фиксировались время старта обработки запроса и время ответа на запрос, после чего вычислялась разница между временами, и полученное значение являлось итоговым временем обработки запроса.

Технические характеристики компьютера, на котором проводились исследования следующие:

1. Процессор – Apple M1 с тактовой частотой 3.2 GHz
2. Объём ОЗУ – 16 ГБ

В данном исследовании изучалась зависимость времени обработки запроса от числа отправляемых электронных писем синхронным и асинхронным методами. Измерения проводились 3 раза, а в качестве данных используется их среднее арифметическое. В результате получены данные, представленные в таблице 22 и таблице 23.

Таблица 22 – время обработки запросов при синхронной отправке писем

Количество электронных писем, шт	10	20	30	40	50	60	70	80	90	100
Время обработки запроса, с	9,08	24,63	39,12	52,80	64,22	75,15	84,97	91,01	101,81	109,19

Таблица 23 – время обработки запросов при асинхронной отправке писем

Количество электронных писем, шт	10	20	30	40	50	60	70	80	90	100
Время обработки запроса, с	0,20	0,22	0,24	0,25	0,29	0,25	0,27	0,30	0,30	0,32

Графики по данным исследования представлены на рисунке 8.

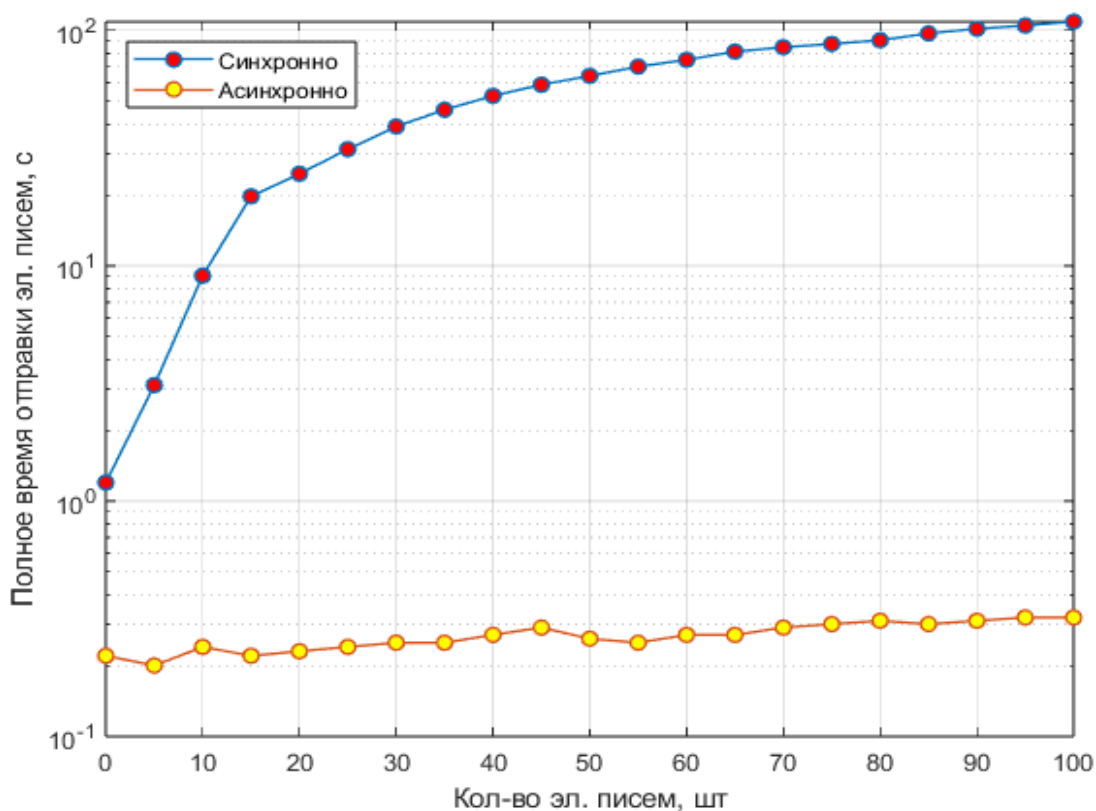


Рисунок 8 – график зависимости времени обработки запроса от количества отправляемых электронных писем синхронно и асинхронно

Исходя из графика зависимости времени обработки запроса от количества синхронно отправляемых электронных писем можно сделать вывод, что время обработки запроса прямо пропорционально количеству отправляемых писем и увеличивается подобно логарифму с увеличением количества отправляемых электронных писем.

Проанализировав график зависимости времени обработки запроса от количества асинхронно отправляемых электронных писем, можно сделать вывод, что количество отправляемых писем слабо влияет на время обработки запросов. С увеличением количества отправляемых писем на порядок, время обработки запроса возросло на 0,1с.

Однако в случае асинхронной отправки данные, представленные в таблице 23 и на графике 8, показывают только скорость обработки запроса к РНР-приложению, но не учитывают время отправки электронных писем, потому что этим занимается РНР-воркер. Время работы РНР-воркера замерялось функцией `microtime()` с момента получения первого сообщения из очереди до момента завершения отправки последнего электронного письма и обновлению информации по отправленному электронному письму в базе данных, результатом являлась разница обозначенных ранее моментов времени. Измерения проводились 3 раза, в качестве данных используется их среднее арифметическое. Результаты представлены в таблице 24.

Таблица 24 – время работы РНР-воркера по обработке электронных писем

Количество электронных писем, шт	1	2	3	4	5	6	7
Время работы РНР-воркера, с	1,878	2,823	4,181	5,642	7,178	8,499	9,071

График зависимости времени отправки электронных писем РНР-воркером от количества электронных писем представлен на рисунке 9.

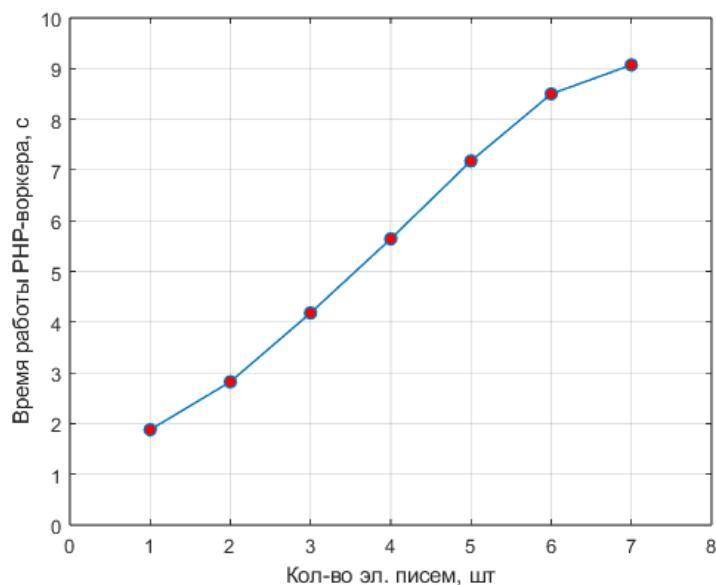


Рисунок 9 – график зависимости времени работы RHP-воркера от количества электронных писем

Исходя из графика видно, что график имеет схожесть с графиком при синхронной отправке электронных писем потому, что RHP-воркер отправляет электронные письма синхронно. Однако с ростом количества электронных писем время их обработки растет быстрее, чем при синхронной отправке. Это связано с тем, что происходят дополнительные запросы в базу данных на поиск и обновление электронного письма, а также с временем получения сообщения из очереди в брокере и его дешифровке.

Полное время отправки электронных писем асинхронным методом будет являться суммой времен обработки запроса и работы RHP-воркера. В таблице 25 представлено полное время отправки электронных писем асинхронным методом.

Таблица 25 – полное время отправки писем асинхронным методом

Количество электронных писем, шт	1	2	3	4	5	6	7
Время обработки запроса, с	2,098	3,036	4,391	5.859	7.395	8.726	9.301

На рисунке 10 представлено итоговое сравнение графиков зависимости полного времени отправки электронных писем синхронным и асинхронным методами отправки.

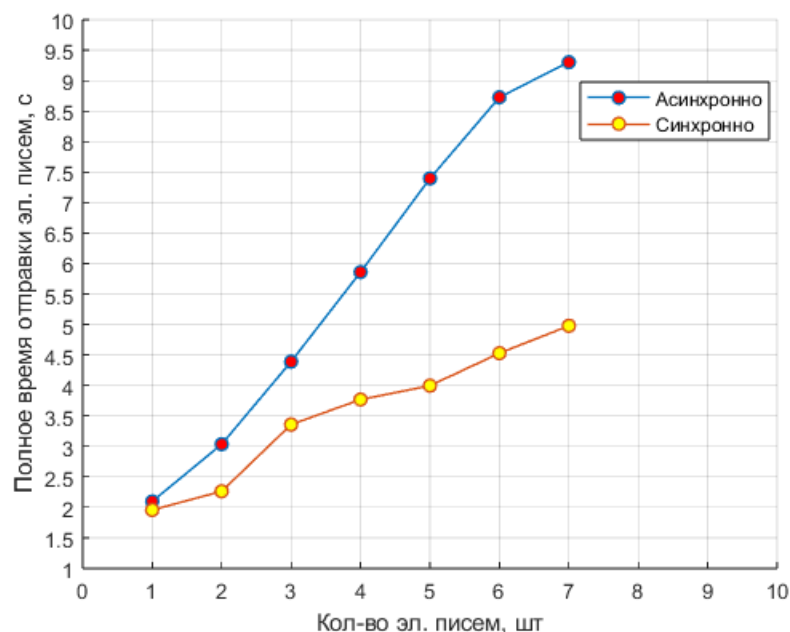


Рисунок 10 – график зависимости полного времени отправки электронных писем синхронным и асинхронным методами

При анализе полученных графиков, можно сделать вывод, что полное время отправки электронных писем синхронным методом увеличивается более медленно, чем время отправки асинхронным методом при увеличении количества отправляемых электронных писем.

4.2 Исследование количества запросов, которое может обработать приложение

В зависимости от выбранного метода отправки электронных писем РНР-приложение может обработать разное количество поступающих запросов в течение определенного количества времени. Это связано с настройкой РНР на сервере, где будет исполняться программный код приложения и сделано для того, чтобы контролировать глобальное количество процессов, которое может быть запущено на сервере.

Для исследования количества запросов к РНР-приложению с отправкой электронных писем, которое может обработано сервером за определенное количество времени был разработан скрипт на языке программирования python [14] с использованием библиотеки locust [15]. Данный скрипт позволит

создать большую нагрузку на приложение и отправлять настраиваемое количество запросов к приложению параллельно. Программный код скрипта представлен в приложении А.

Измерения проводились 3 раза, в качестве данных используется их среднее арифметическое. Был выбран промежуток времени равный 60 секундам. На рисунке 11 представлено итоговое сравнение количества запросов по отправке 5, 10, 15 и 20 электронных писем синхронным и асинхронным методами, которое может быть обработано сервером в течение 60 секунд.

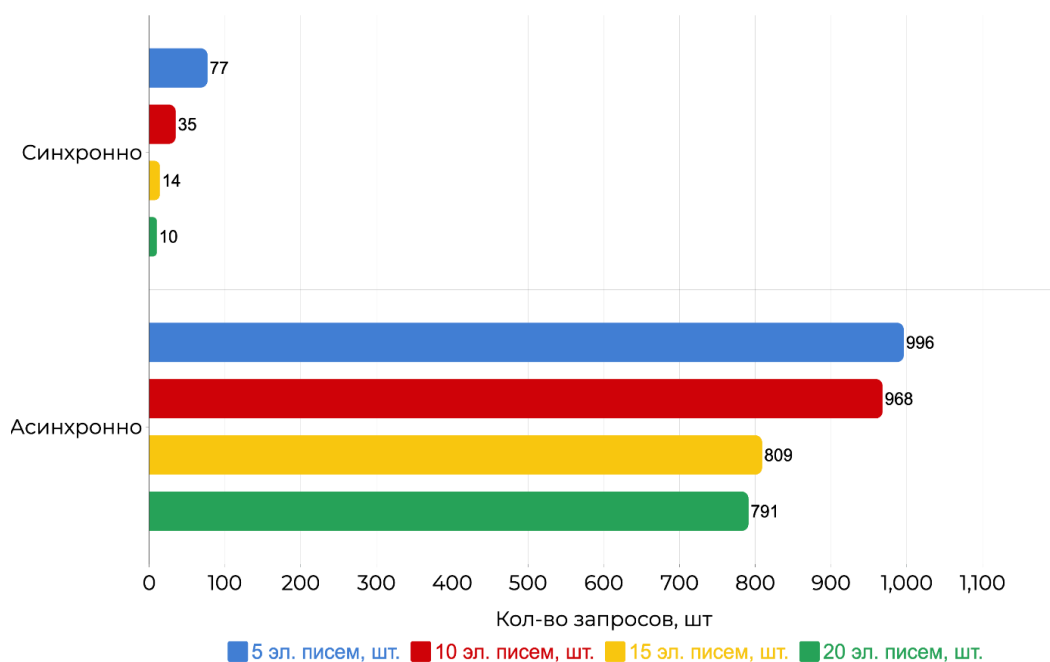


Рисунок 11 – сравнение количества запросов, которое может обработать приложение при отправке различными методами

Как видно из рисунка, при отправке электронных писем асинхронным методом, количество запросов, которое может обработать приложение за 60 секунд в несколько десятков раз больше, чем при отправке писем синхронно. Для 5, 10, 15 и 20 электронных писем, отправляемых в рамках одного запроса.

4.3 Исследование увеличения количества РНР-воркеров

С ростом нагрузки на РНР-приложение подразумевается большее количество адресованных к нему пользовательских запросов. Это означает, что наполняемость очереди по отправке электронных писем будет увеличиваться.

В таком случае возможно оказаться в ситуации, когда электронные письма отправляются с ощутимой задержкой во времени. Данное явление связано с тем, что РНР-воркер синхронно отправляет каждое электронное письмо в соответствии с его позицией в очереди. Для решения данной проблемы возможно осуществить увеличение количества РНР-воркеров по обработке очереди по отправке электронных писем.

В таблице 26 представлено время обработки всех сообщений в очереди брокера одним РНР-воркером. Измерения проводились 3 раза, а в качестве данных используется среднее арифметическое измерений.

Таблица 26 – время обработки сообщений одним РНР-воркером

Количество сообщений в очереди, шт	Время обработки всех сообщений, с
10	12,281
100	110,889
500	488,025
1000	1073,300

Как видно из таблицы 26, обработка 1000 сообщений, накопленных в очереди, занимает 1073,300 секунд, что является очень большим показателем. Поэтому возникает необходимость добавить дополнительных РНР-воркеров для уменьшения времени обработки всех сообщений очереди.

В таблице 27 представлено время обработки всех сообщений в очереди брокера двумя РНР-воркерами. Измерения проводились 3 раза, а в качестве данных используется среднее арифметическое измерений.

Таблица 27 – время обработки сообщений двумя РНР-воркерами

Количество сообщений в очереди, шт	Время обработки всех сообщений, с
10	5,778
100	57,391
500	232,990
1000	493,501

Как видно из таблицы 27, добавление второго РНР-воркера помогло сократить время обработки всех сообщений заполненной очереди примерно в два раза.

4.4 Выводы по итогам исследования разработанного метода

По итогам проведенного исследования разработанного метода можно сделать следующий вывод: разработанный метод асинхронной отправки позволит увеличить скорость ответа РНР-приложений на запросы, при обработке которых необходимо отправить электронные письма, что положительно скажется на производительности приложения и пользовательском опыте клиентов, использующих это веб-приложение, потому что скорость обработки запросов составляет порядка 0,32 секунды при отправке 100 электронных писем в рамках обрабатываемого запроса. Однако время полной отправки электронных писем прямо пропорционально количеству отправляемых писем и заметно возрастает с их увеличением.

Для уменьшения полного времени обработки всех сообщений в очереди по отправке электронных писем возможно увеличить количество обработчиков очереди, а именно РНР-воркеров. Благодаря увеличению количества РНР-воркеров в два раза получится сократить время обработки всех сообщений очереди тоже в два раза.

Также было исследовано количество запросов на отправку электронных писем, которое может обработать РНР-приложение за 60 секунд, при отправке писем синхронным и асинхронным методами. Исследование показало, что при использовании асинхронного метода электронных писем, РНР-приложение может обработать в несколько десятков раз больше запросов, чем при использовании синхронного метода отправки электронных писем.

Таким образом, можно говорить о том, что разработанный метод не только повысит производительность приложения, но и может быть масштабирован путем добавления новых обработчиков той же очереди в брокере сообщений, что позволит приложению обрабатывать большие объемы данных без дополнительной оптимизации разработанного метода.

5 ОБЕСПЕЧЕНИЕ КАЧЕСТВА РАЗРАБОТКИ, ПРОДУКЦИИ, ПРОГРАММНОГО ПРОДУКТА

5.1 Определение потребителей

Конечными потребителями разработанного «Метода асинхронной отправки электронных писем в РНР-приложениях» являются менеджеры, оформляющие командировки для корпоративных клиентов компании ООО «Центр Интеграции Приложений» в онлайн-системе бронирования туристических услуг Colibri.travel, а также сами путешественники. Потребители получают на электронную почту письма, формируемые и отправляемые веб-приложением. Данный метод нацелен на сокращение времени ожидания ответов веб-приложения на запросы пользователей, увеличение надежности доставки и повышение гибкости настройки электронных писем. Заинтересованной стороной разработанного метода является руководство компании, так как внедрение метода позволит улучшить пользовательский опыт менеджеров и повысить лояльность клиентов компании.

5.2 Функции продукции

Перед тем, как начать выделять функции разработанного метода необходимо дать точное определение того, что подразумевается под функцией продукта в данном случае. Функции изделия или услуги – это требования и ожидания потребителя, которые могут быть установлены, предполагаются или являются обязательными.

Для разработанного метода были выделены следующие функции:

- Возможность формирования тела отправляемого письма с использованием HTML-разметки для повышения читаемости важных элементов наполнения письма путем их выделения.
- Возможность добавления файлов вложений различных форматов к электронному письму.
- Возможность указания переменного числа электронных адресов первичных и вторичных получателей письма.

- Возможность сохранения в базе данных полной информации об отправляемом электронном письме.
- Возможность уточнения статуса отправки электронного письма.
- Возможность повторной переотправки электронного письма в случае возникновения ошибок при отправке.

5.3 Качество и характеристики

Согласно ГОСТ Р ИСО 9000-2015 [16], качество – это степень соответствия совокупности присущих характеристик объекта требованиям. Был проведен анализ стандартов, используемых в сфере разработки программного обеспечения, для обеспечения общего уровня качества разрабатываемого метода.

Применение установленных протоколов, стандартов и регулирующих актов, принятых в данной сфере, обеспечивает высокий уровень качества и соответствие продукта актуальным стандартам и современным требованиям.

Анализ функциональных требований к разработке на основании выбранного стандарта ГОСТ Р ИСО/МЭК 25010-2015 [17] представлен в таблице 28.

Таблица 28 – функциональные требования

Функции по группам	Источник	Требование
Удовлетворенность	ГОСТ Р ИСО/МЭК 25010-2015	Электронные письма, отправляемые веб-приложением, должны приходить на почту как менеджерам, создавшим командировку, так и путешественникам этой командировки

Продолжение таблицы 28

Конфиденциальность	ГОСТ Р ИСО/МЭК 25010-2015	Содержимое электронных писем должно подвергаться шифрованию на пути от отправителя к получателю дабы избежать потери конфиденциальной информации в случае перехвата
Надёжность	ГОСТ Р ИСО/МЭК 25010-2015	В случае ошибки отправки электронного письма веб-приложение должно отобразить сообщение об ошибке
Использование ресурсов	ГОСТ Р ИСО/МЭК 25010-2015	При использовании метода веб-приложение должно расходовать ограниченное и прогнозируемое количество оперативной памяти и дискового пространства сервера
Производительность	ГОСТ Р ИСО/МЭК 25010-2015	Разработанный метод должен обеспечить максимальную производительность при отправке изменяемого количества электронных писем в рамках обработки пользовательского запроса к веб-приложению

5.4 Измерение характеристик качества. Операционное определение

Операциональное определение (ОО) – это уточнение значения того или иного термина применительно к данной системе, находящейся в конкретных условиях и для людей, в ней задействованных.

ОО позволяет решить сразу 2 задачи: присвоить термину точный смысл для формирования «общего языка» между людьми и исключения возможности разночтения, определение контекста (окружения), в котором это слово используется.

ОО содержит как минимум три компоненты:

1. Требования или стандарт, относительно которого оценивается результат измерения или испытания (критерий).
2. Метод испытания или процедура измерения свойства объекта (тест).
3. Процедура принятия решения (анализ), которое показывает, соответствует ли результат испытания стандарту.

Операционные определения для разработанного метода представлены в приложении Б в таблице 29.

5.5 Предложения по улучшению

По итогам рассмотрения ОО было определено, что разработанный и внедренный в веб-приложение Colibri.travel метод удовлетворяет большинству описанных требований, однако существует требование, которое удовлетворяется частично. Предложения по улучшению метода сформулированы в таблице 30.

Таблица 30 – предложения по улучшению метода

Требование	Анализ текущего состояния	Возможные причины невыполнения критерия	Предложения по улучшению
При использовании метода веб-приложение должно расходовать ограниченное и прогнозируемое количество оперативной памяти и дискового пространства сервера	Линейный характер роста	Недостаточно нормализованная таблица для хранения информации по электронным письмам в базе данных и неограниченное количество подключений к базе данных из веб-приложения	Провести дополнительную нормализацию спроектированной таблицы в базе данных, используя нормальные формы, а также спроектировать и реализовать внутреннее программное решение, позволяющее установить переиспользуемое соединение с базой данных из веб-приложения

5.6 Выводы

Были определены целевые пользователи продукта, а также описаны основные функции конечного продукта. После анализа стандартов, отраслевых требований и современных лучших практик, необходимых для проектирования и разработки метода, были сформулированы функциональные требования, а также были описаны операциональные определения. В результате исследования было установлено, что разработанный метод соответствует большинству указанных требований, но одно из них выполняется частично.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была достигнута поставленная цель – был разработан метод асинхронной отправки электронных писем для повышения производительности отправки электронных писем в РНР-приложении Colibri.travel, а также были достигнуты все поставленные задачи:

1. Исследовать способы использования асинхронного программирования в РНР.
2. Рассмотреть асинхронные методы отправки электронных писем в РНР-приложениях.
3. Разработать архитектуру асинхронного метода отправки электронных писем.
4. Реализовать асинхронный метод отправки электронных писем.
5. Исследовать разработанный метод.

Были сформулированы критерии оценивания аналогов, а также были отобраны сами аналоги для сравнительного обзора. По итогам обзора был сделан вывод, что в полной мере ни один из представленных аналогов не решает поставленную задачу. На основании сформулированных критериев сравнения аналогов, а также их сильных и слабых сторон, была проведена формулировка требований к решению и определена постановка задачи.

Была разработана архитектура метода асинхронной отправки электронных писем в РНР-приложениях. Общий алгоритм работы метода был логически поделен на этапы, далее для выполнения каждого из этапов был реализован при помощи отдельного класса или набора классов. Такой подход позволил не только сократить время разработки метода, но и предусмотреть простое масштабирование метода.

На основании разработанной архитектуры был создан сам метод, решающий поставленную задачу и отвечающий соответствующий всем требованиям из перечня.

Было проведено исследование инструмента на предмет производительности, а именно были исследованы и сравнены время отправки электронных

писем синхронным и асинхронным методами, исследовано количество запросов, которое может быть обработано приложением за определенный промежуток времени при отправке электронных писем синхронно и асинхронно и исследовано увеличение количества РНР-воркеров для масштабирования разработанного метода. По итогам исследования был сделан вывод о том, что отправка электронных писем в рамках обработки запроса к приложению асинхронным методом происходит значительно быстрее, чем синхронным, чтократно уменьшает время ответа РНР-приложения тем самым увеличивая его производительность. Данные исследования свидетельствуют о том, что реализованный асинхронный метод по отправке электронных писем не нуждается в дополнительной оптимизации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. «Tiobe Index for April 2024» [Электронный ресурс]. URL: <https://www.tiobe.com/tiobe-index/> (Дата обращения 05.04.2024).
2. «Usage statistics of server-side programming languages for websites» [Электронный ресурс]. URL: https://w3techs.com/technologies/overview/programming_language (Дата обращения 05.04.2024).
3. Федоров, М. В. Сравнительный анализ двух языков программирования / М. В. Федоров // Научно-исследовательский центр «Technical Innovations». – 2023. – № 10. – С. 196-201. -EDN EIGGZC.
4. Любимова, Т. В. Асинхронность в программировании / Т. В. Любимова // Университетская наука. – 2019. – № 2(8). – С. 135-138. – EDN BJWENB.
5. «Exec function manual» [Электронный ресурс]. URL: <https://www.php.net/manual/ru/function.exec.php> (Дата обращения 05.04.24).
6. «Fibers manual» [Электронный ресурс]. URL: <https://www.php.net/manual/ru/language.fibers.php> (Дата обращения 05.04.24).
7. «AMPHP documentation» [Электронный ресурс]. URL: <https://amphp.org/> (Дата обращения 05.04.24).
8. «What is a message broker?» [Электронный ресурс]. URL: <https://www.ibm.com/topics/message-brokers> (Дата обращения 05.04.24).
9. Семенов, М. В. Kafka и rabbitmq: сравнительный анализ сервисов обработки очередей / М. В. Семенов, Н. А. Шиндяпкин // Вестник коломенского института (филиала) Московского политехнического университета: Сборник научных трудов. – Москва: федеральное государственное бюджетное образовательное учреждение высшего образования "Московский политехнический университет", 2021. – С. 248-266. – EDN UFAWZZ.
10. «Mail function manual» [Электронный ресурс]. URL: <https://www.php.net/manual/ru/function.mail.php> (Дата обращения 05.04.24).

11. Кочитов, М. Е. Использование библиотеки SwiftMailer для отправления писем на электронную почту / М. Е. Кочитов // Постулат. – 2020. – № 8(58). – С. 8. – EDN CSDPRD.
12. Laravel PHP framework and project created using Laravel // International Journal of Progressive Research in Engineering Management and Science. – 2023. – DOI 10.58257/ijprems30976. – EDN QBCLHW.
13. «Php-amqp-lib documentation» [Электронный ресурс]. URL: <https://github.com/php-amqp-lib/php-amqp-lib> (Дата обращения 05.04.24).
14. «Python documentation» [Электронный ресурс]. URL: <https://www.python.org/> (Дата обращения 05.04.24).
15. «Locust documentation» [Электронный ресурс]. URL: <https://locust.io/> (Дата обращения 05.04.24).
16. ГОСТ Р ИСО 9000-2015.
17. ГОСТ Р ИСО/МЭК 25010-2015.

ПРИЛОЖЕНИЕ А

Файл locustfile.py

```
from locust import HttpUser, task
class HelloWorldUser(HttpUser):
    @task
    def hello_world(self):
        self.client.get("/api/sync-email-send")
        self.client.get("/api/async-email-send")
```

ПРИЛОЖЕНИЕ Б

Таблица 29 – операциональные определения

Требование	Тест		Решение (правило)		
	Характеристика	Метод измерения	Соответствие	Частичное соответствие	Несоответствие
Электронные письма, отправляемые веб-приложением, должны приходить на почту двум группам пользователей (менеджеры и путешественники)	Доля успешно доставленных писем, %	Отправка тестовых писем в количестве 100 штук каждой из групп пользователей и подсчет доли успешно доставленных писем	Электронные письма приходят в полном объеме обеим группам пользователей (100% для каждой группы)	Электронные письма приходят в неполном объеме одной из групп пользователей (менее 100% для одной из группы пользователей)	Электронные письма не приходят в полном объеме обеим группам пользователей (менее 100% для каждой из групп пользователей)
В случае возникновения ошибки отправки электронного письма веб-приложение должно отобразить сообщение об ошибке	Доля обработанных электронных писем, при отправке которых возникла ошибка, %	Отправка тестовых писем с неправильным электронным адресом в количестве 100 штук и подсчет доли обработанных писем	100%	95%	Менее 95%

Продолжение таблицы 29

Содержимое электронных писем должно подвергаться шифрованию на пути от отправителя к получателю дабы избежать потери конфиденциальной информации в случае перехвата	Процент зашифрованных перед отправкой электронных писем	Подсчёт количества зашифрованных электронных писем относительно всех электронных писем	100%	90%	Менее 90%
При использовании метода веб-приложение должно расходовать ограниченное и прогнозируемое количество оперативной памяти и дискового пространства сервера	Объем занимаемой оперативной памяти и дискового пространства сервера	Исследование с отправкой заданного количества электронных писем в рамках одного запроса к веб-приложению	Существует верхний предел траты оперативной памяти и дискового пространства сервера, не зависящий от числа отправляемых электронных писем	Линейный характер роста	Квадратичный или кубический характер роста

Продолжение таблицы 29

Разработанный метод должен обеспечить максимальную производительность при отправке измененияемого количества электронных писем в рамках обработки пользовательского запроса к веб-приложению	Время обработки запроса на отправку электронных писем	Измерение среднего значения времени, которое требуется на обработку запроса к веб-приложению по отправке электронных писем. Выборка формировалась из 10 запросов по отправке 10 электронных писем в каждом	1 секунда	От 1 до 3 секунд	Более 3 секунд
--	---	--	-----------	------------------	----------------

ПРИЛОЖЕНИЕ В



Общество с ограниченной ответственностью
«Центр интеграции приложений»

123007, город Москва, ш. Хорошёвское, дом 32А
Тел. (495) 980-73-23

ОКПО: 38309349, ОГРН: 1127746060056
ИНН/КПП: 7714863245/771401001

АКТ

о внедрении результатов дипломной работы

Денежного Анатолия

на тему:

«Разработка асинхронного метода отправки электронных писем в РНР-приложениях»

Настоящий акт составлен о том, что результат выпускной квалификационной работы студента СПбГЭТУ «ЛЭТИ» группы 0303 очной формы обучения Денежного Анатолия на тему «Разработка асинхронного метода отправки электронных писем в РНР-приложениях» внедрен в онлайн-систему бронирования туристических услуг для корпоративных клиентов ООО «Центр интеграции приложений».

Внедрение разработанного метода в продукт Colibri.travel позволило сократить время ожидания ответа от приложения на пользовательский запрос в случаях, когда необходимо отправить электронное письмо на почту за счёт параллельной обработки пользовательского запроса и процесса отправки электронного письма.

Генеральный директор

ООО «Центр интеграции приложений»

«02» 04 2024г.



А.В. Марченко